Vim Cheat Sheet

Vim can be controlled entirely from the keyboard, making it possible to work much more efficiently than in many other editors. Of course, you can still use the mouse, but Vim really shows its strengths when operated from the keyboard alone.

This is possible because Vim has different modes that you switch between. The three most important ones are shown in the diagram below (Figure 1).

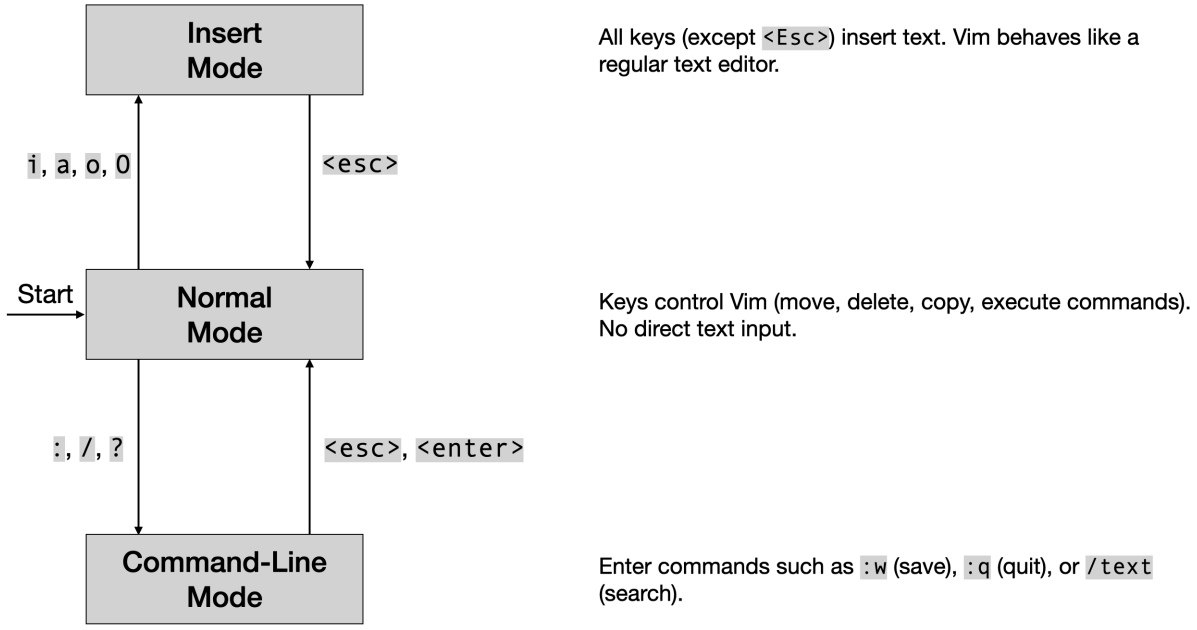

Figure 1: State diagram of the three most important Vim modes: Normal, Insert, and Command-Line.

Vim actually has more modes than those shown here, but these three are the foundation. We will introduce additional modes later if and when we need them.

Important: No matter which mode you are currently in, pressing `<Esc>` always takes you back to Normal mode.

Rule of thumb: When in doubt, press `<Esc>` one more time. Then you always know where you are.

Starting Vim[1]

In most cases, you specify the file you want to edit when starting Vim. For example:

```
vim foo.abc
```

This opens the file `foo.py` or creates it if it does not already exist.
You can also start Vim without specifying a file:

```
vim
```

In that case, it is useful to know a few basic command-line commands in order to:
- save your work under a filename later, or
- open an existing file from within Vim.

---

[1] During the HPC0 installation session (Session 00), we configured `vim` as an alias for `nvim`, the actual Neovim executable. If you prefer, you are welcome to type `nvim` directly or remove the alias altogether.

Important Command-Line Commands

Enter these commands after pressing `:` in Normal mode.

Command	Meaning
`:w`	Save the current file *(write)*
`:q`	Quit Vim, provided the current file has been saved
`:q!`	Quit Vim without saving
`:wq`	Save and quit
`:w <filename>`	Save under a different filename
`:e <filename>`	Open another file *(edit)*
`:e#`	Return to the previously opened file
`:1`	Jump to line 1 (in general: `:<number>` jumps to line `<number>`)
`:$`	Jump to the last line

Additional HPC0 Configuration

The HPC0 Neovim configuration comes with a few plugins that make editing more convenient.

For example, it includes a file explorer that can be toggled with:

- `Space + e` → show or hide the file explorer

Navigation between windows and tabs has also been simplified:

- `Alt` + `Arrow Keys` → switch between editor windows
- `Shift` + `Arrow Keys` → switch between tabs

Instead of the arrow keys, you can also use `h`, `j`, `k`, and `l` for navigation. At this point, you probably won't understand why anyone would want to do that—but give it a week, and you'll most likely find yourself reaching for the arrow keys much less often.

These are not standard Vim commands, but they make working with Vim much easier during the course.

Important Commands in Normal Mode

1. Entering Insert Mode

Command	Meaning
`i`	Insert text before the cursor *(insert)*
`a`	Insert text after the cursor *(append)*
`o`	Open a new line below the current line and enter Insert mode
`O`	Open a new line above the current line and enter Insert mode

2. Copy and Paste (Yank & Put)

Command	Meaning
`yy`	Yank (copy) the current line
`2yy`	Yank (copy) two lines (in general: `<number>yy`)
`p`	Put (paste) below the current line or after the cursor
`P`	Put (paste) above the current line or before the cursor

3. Navigation Within the Current Line

Command	Meaning
`0`	Move to the beginning of the current line
`^`	Move to the first non-whitespace character
`$`	Move to the end of the current line
`w`	Move to the beginning of the next word
`b`	Move to the beginning of the previous word
`e`	Move to the end of the current word

4. Deleting and Changing (Essential Commands)

Command	Meaning
`dd`	Delete the current line
`2dd`	Delete two lines (in general: `<number>dd`)
`D`	Delete from the cursor to the end of the line
`dw`	Delete from the cursor to the beginning of the next word
`2dw`	Delete two words (in general: `<number>dw`)
`cw`	Change a word (delete it and enter Insert mode)
`2cw`	Change two words (in general: `<number>cw`)
`cl`	Change one character (delete it and enter Insert mode)
`2cl`	Change two characters (in general `<number>cl`)
`x`	Delete the character under the cursor (in general: `<number>x`)
`X`	Delete the character before the cursor (in general: `<number>X`)

Note:

Anything deleted or changed with commands such as `dd`, `dw`, or `cw` is automatically stored in one of Vim's internal registers. You can insert the stored text with `p` (put) below the current line or after the cursor, and with `P` above the current line or before the cursor. In this sense, deleting and changing in Vim work much like Cut in other editors—but they are considerably more flexible. Try to figure out what `yw` or `2yw` do before looking them up.

5. Undo / Redo / Repeat

Command	Meaning
`u`	Undo the last change
`Ctrl-r`	Undo the last change
`.`	Repeat the last change

6. Text Objects (A First Look)

Command	Meaning
`ct(`	*Change to* `(` – *change everything up to the next* `(`
`dt(`	*Delete to* `(` – delete everything up to the next `(`
`ct"`	*Change to* `"` – change everything up to the next `"`
`dt"`	*Delete to* `"` – delete everything up to the next `"`
`ci(`	*Change in* `(` – change everything inside the current (or next) pair of parentheses
`di(`	*Delete in* `(` – delete everything inside the current (or next) pair of parentheses
`ci"`	*Change in* `"` – change everything inside the current (or next) pair of quotation marks
`di"`	*Delete in* `"` – delete everything inside the current (or next) pair of quotation marks

Tip: Try commands such as `ciw`, `ci'`, `ci{`, `ci[`, or `ci<`. See if you can figure out which text object each command operates on.

7. Indenting and Joining Lines

Command	Meaning
`J`	Join the current line with the next line
`>>`	Indent the current line to the right
`2>>`	Indent two lines to the right (in general: `<number>>>`)
`<<`	Indent the current line to the left
`2<<`	Indent two lines to the left (in general: `<number><<`)

8. Navigation with h, j, k, and l

Note: This style of navigation is especially efficient if you know how to touch type. For beginners, the arrow keys work perfectly well. The keys `h`, `j`, `k`, and `l` become truly useful only after some practice.

Command	Meaning
`h`	Move one character left
`l`	Move one character right
`j`	Move one line down
`k`	Move one line up

Try it: What do you think the commands `3j` or `2h` do? Give them a try! Most Vim commands can be prefixed with a number to repeat them multiple times.

A Final Remark

As mathematicians, we have learned to wield one of our greatest weapons: **the mighty sword of deduction**.

Instead of memorizing long lists of rules, we look for a small set of underlying principles from which the apparent special cases naturally follow. Vim is no exception. Most of its commands are built from a surprisingly small number of concepts that can be combined in systematic ways.

However, mathematics teaches us a second lesson as well.

Although you could derive the rules for fraction arithmetic from the axioms every single time, nobody actually does. Those rules eventually become second nature, leaving your mind free to focus on the interesting part of the problem.

The same is true for Vim.

At first, you will consciously think about every command. But if you use Vim regularly, the most common ones will eventually move into your "muscle memory". Once that happens, you can stop thinking about the editor and focus entirely on your code.

So don't try to memorize this cheat sheet.

Use it. Look things up. Repeat. The memorization will happen automatically.